¡hola! · workshop · DataSaturday Madrid

Dime qué commiteas

y te diré quién eres

Git para gente de Power BI y Microsoft Fabric

Viernes 6/11 · 10:00–14:00

Todo se puede deshacer



Mar



Adrián



Raquel

Quiénes somos

Hoy os acompañamos tres personas: preguntad a cualquiera



Mar Lizana Atienza

Data Squad Lead @ ENCAMINA

Microsoft MVP · Fabric y DevOps





Adrián del Rincón

Data Architect @ ENCAMINA

Microsoft, Databricks y Gen AI



Raquel Rega

Data Engineer @ ENCAMINA

Primero, dejamos todo listo

10:00–10:15 · nadie arranca hasta que estemos todas

1 Tu usuario

Entra en **app.fabric.microsoft.com** con el usuario y la contraseña de tu tarjeta, y cámbiala.

2 MFA

Registra Microsoft Authenticator en el móvil: escanea el QR de la pantalla.

3 Trial de Power BI Pro

En Fabric, pulsa tu foto (arriba a la derecha) y derecha) y elige «Iniciar prueba gratuita».

4 GitHub

¿Tienes cuenta? Inicia sesión. ¿No? La creamos creamos juntas en **github.com/signup**.

El plan de la mañana

Hora	Bloque	Ejercicio
10:00–10:15	Usuario, MFA, trial de Pro y cuenta de GitHub	—
10:15–10:30	Repo, commit, rama, merge y pull request	—
10:30–10:50	Conectar tu workspace a GitHub	1 y 2
10:50–11:05	Las tripas: PBIP y TMDL	3
11:05–11:30	Ramas, workspaces y pull requests	4
11:30–12:05	**Estrategias de Git y de ramas**	**5**
12:05–12:20	**Descanso**	—
12:20–12:50	El repo común: conflicto y revert	6 y 7
12:50–13:35	**Deployment pipelines**	**8**
13:35–13:50	De dev a prod: las cuatro opciones y la escalera de madurez	Demo
13:50–14:00	Novedades y cierre	—

¿Cuántas versiones de este informe tienes?

ventas.pbix

ventas_v2.pbix

ventas_FINAL.pbix

ventas_FINAL_bueno.pbix

ventas_FINAL_bueno_ESTE.pbix

¿Qué cambió?

¿Quién lo cambió?

¿Por qué?

Git guarda cada cambio con autor, autor, fecha y mensaje. Y te deja deja volver atrás.

Seis palabras y ya hablas Git

Todo lo demás son detalles

Repo

El historial completo de tu proyecto.

Commit

Una foto de tus cambios con un un mensaje: qué y por qué.

Rama

Una línea de trabajo paralela que que no toca lo que funciona.

Merge

Juntar los cambios de una rama en otra.

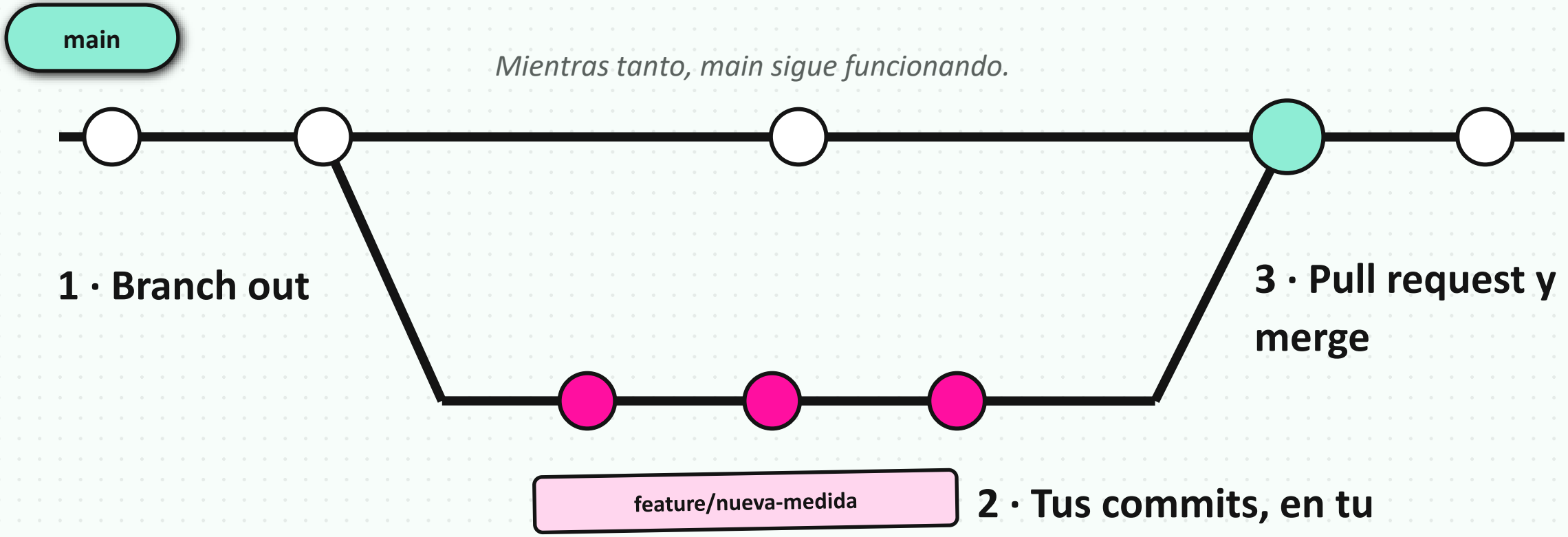
Pull request

«Quiero meter mi rama en main, ¿lo revisas?»

main

La rama principal: lo que funciona. Nadie trabaja en ella directamente.

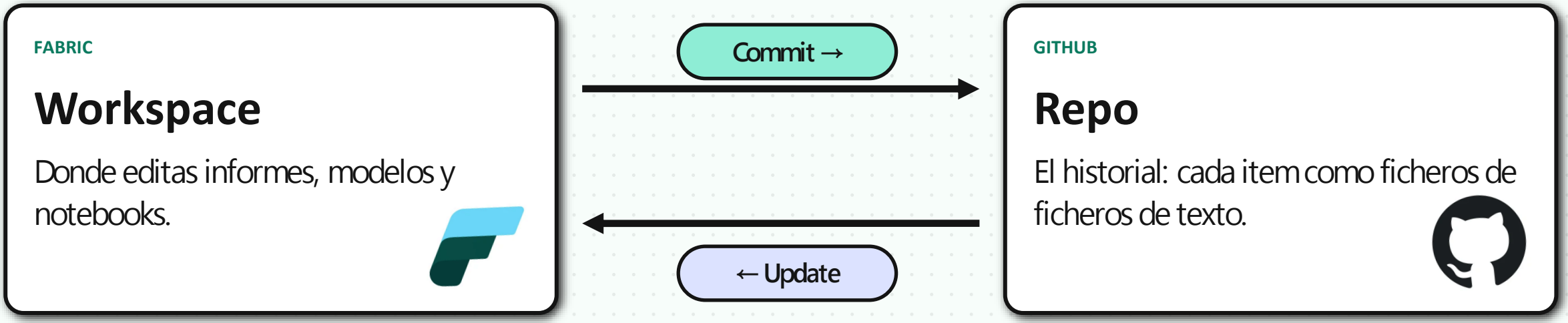
Una rama, dibujada



Cada punto es un **commit**. La rama sale de main, recibe tus cambios y vuelve con un **merge**.

Tu workspace y tu repo, conectados

Fabric no tiene Git dentro: guarda una copia en texto



Un workspace se conecta a **una rama**. **Commit** manda tus cambios a Git; **Update** trae a Fabric lo que cambió en que cambió en Git.

Guardar no es hacer commit. Guardar deja el cambio en Fabric. Hasta que pulsas Commit, Git no Git no sabe nada.

Conectar un workspace

Workspace settings → Git integration

1

Proveedor

GitHub o Azure DevOps.

2

Cuenta

Un token: la llave que le
le das a Fabric para entrar
entrar en tu repo, y solo
solo en ese.

3

Repo y rama

El repo y la rama,
normalmente main.

4

Carpeta

Dónde van los items: / o
o una subcarpeta.

Solo un Admin del workspace puede conectarlo, y el workspace tiene que estar en una capacidad de capacidad de Fabric.

Qué te dice cada item

Fabric no sube ni baja nada hasta que tú lo pides

Estado	Qué ha pasado	Qué haces
Synced	Igual en Fabric y en Git	Nada
Uncommitted	Lo cambiaste en Fabric	Commit
Update required	Alguien lo cambió en Git	Update
Conflict	Cambió en los dos sitios	Elegir versión

El panel Source control

Lo agrupa en dos pestañas: Changes, lo que vas a subir, y Updates, lo que vas a traer.

MANOS A LA OBRA · 15 MIN

Ejercicios 1 y 2: tu repo y tu primer commit

1. En GitHub, crea el repo privado **workshop-git**, con README.
2. Crea un token fine-grained: solo ese repo, Contents de lectura y escritura.
3. Conecta **ws-gitws-userN** a tu repo: rama main, carpeta /.
4. Crea un lakehouse y un informe.
5. Haz commit con un mensaje que diga qué has hecho y por qué.
6. Abre GitHub y mira qué ficheros han aparecido.

Paso a paso: README del repo gitws-team

Un informe en Git es una carpeta

El mismo formato que el proyecto de Power BI Desktop (PBIP)

tu repo

```
Ventas.Report/
  definition.pbir
  definition/
  .platform
Ventas.SemanticModel/
  definition.pbism
  definition/
    model.tmdl
    tables/
      Ventas.tmdl
  .platform
```

.Report

Lo que se ve: páginas, visuales y filtros.

.SemanticModel

El modelo: tablas, medidas y relaciones.

.platform

El carné del item: tipo, nombre e ID lógico.

TMDL: el modelo, en texto legible

Una tabla por fichero, indentado y sin sorpresas

Ventas.tmdl

```
table Ventas
  measure 'Total ventas' = SUM(Ventas[Importe])
-   formatString: #,##0
+   formatString: #,##0.00 €
  column Importe
    dataType: decimal
```

El diff es una línea

Cambias el formato de una medida y medida y Git te enseña exactamente exactamente eso.

Con un .pbix…

Git solo sabe que «ha cambiado algo». algo». Con TMDL ves qué, y lo revisas revisas antes de que llegue a producción.

MANOS A LA OBRA · 10 MIN

Ejercicio 3: mira las tripas

1. En GitHub, abre **.SemanticModel/definition**: cada tabla es un fichero .tmdl.
2. En Fabric, cambia el formato de una medida.
3. Haz commit con un mensaje que lo explique.
4. En GitHub, abre el commit y mira el diff: ese es el cambio, y nada más.

Paso a paso: README del repo gitws-team

En main no se pinta

Los errores, en tu rama, donde salen baratos

main, protegida

Lo que funciona. Solo entra por por pull request.

Una rama por cambio cambio

feature/nueva-medida, fix/filtro-fecha: corta y con un nombre que se entienda.

Branch out

Fabric crea la rama y puede darte un workspace solo para para ella.

Lo ideal: un workspace por persona y rama. Así nadie pisa el trabajo de nadie mientras prueba.

Ramas y workspaces: tres formas de repartirlos

No hay una buena para todos: depende del tamaño del equipo

COMPARTIDO

Uno por entorno

dev, test y prod. Todo el equipo trabaja en dev, conectado a una una rama.

+ Sencillo de montar.

– Os pisáis: lo que está a medias en medias en dev lo ve todo el mundo. mundo.

POR PERSONA

Uno por persona

Cada una con su rama y su workspace, creados con Branch out.

+ Pruebas aisladas, sin miedo.

– Más workspaces que mantener y mantener y más capacidad.

POR FEATURE

Uno por cambio

Vive lo que dura la rama y se borra al hacer merge.

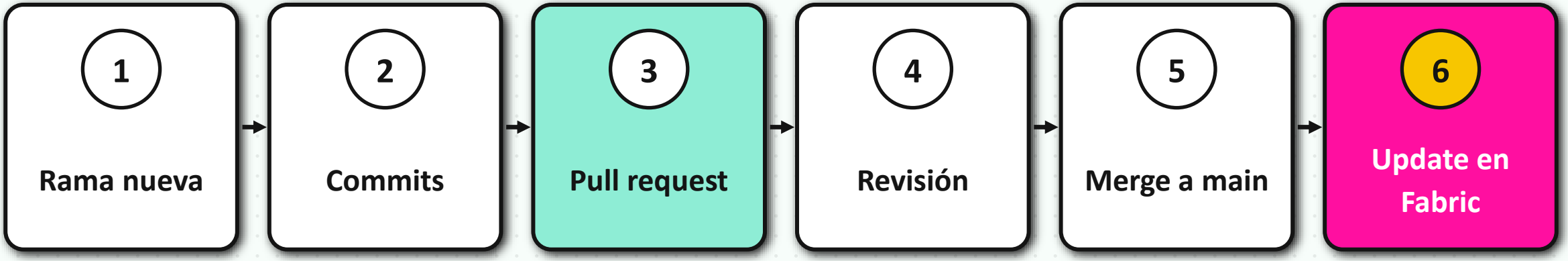
+ Limpio, y la revisión es de verdad. verdad.

– Hay que automatizarlo: Branch Branch out, la API o fabric-cicd.

Novedad: Fabric ya recuerda de qué workspace sale cada rama (**branch workspaces**, GA).

Fuente: Tabular Editor, «Building CI/CD maturity for report development» (octubre 2026)

El viaje de un cambio



La revisión no es desconfianza

Es leer el diff antes de que llegue a producción. producción.

El paso 6 se olvida siempre

El workspace conectado a main no se actualiza solo. solo.

MANOS A LA OBRA · 15 MIN

Ejercicio 4: tu primera pull request

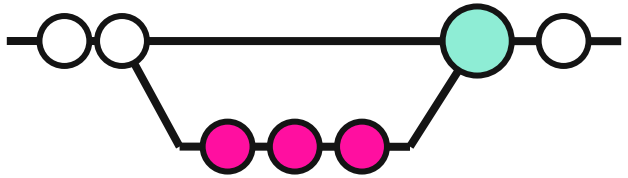
1. **Branch out** a una rama nueva: **feature/tu-nombre**.
2. Cambia algo en el informe y haz commit en esa rama.
3. En GitHub, abre una pull request hacia main y lee el diff.
4. Haz merge: en tu propio repo no hay aprobación (GitHub no deja aprobar tu PR).
5. Vuelve al workspace de main y pulsa **Update all**: ahí está tu cambio.

Paso a paso: README del repo gitws-team

Tres formas de juntar una rama

El botón verde de la pull request tiene un desplegable

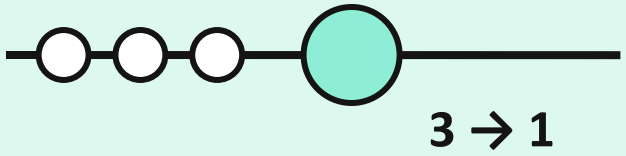
Merge commit



Crea un commit que junta las dos ramas. Se ven todos tus commits y y dónde se unen.

Para promocionar entre ramas largas: largas: dev → main.

Squash and merge



Aplasta todos los commits de la rama rama en uno solo, encima de main. main.

Para las features: un cambio, un commit. Revertirlo es un clic.

Rebase and merge



Copia tus commits uno a uno encima encima de main, sin commit de merge. merge.

Historial en línea recta, pero reescribe tus commits. Para más adelante.

Nuestra receta: squash para las features y merge commit para promocionar entre ramas largas.

Estrategias de ramas: las tres familias

Cuántas ramas largas tienes y cuánto viven las cortas

GITHUB FLOW

main y ramas cortas

main + **feature/***. Cada cambio en su rama, PR a main y main siempre lista para desplegar.

+ Sencilla y encaja con Fabric.

– Test y prod salen de main: hace falta falta algo más, como pipelines o tags. tags.

GITFLOW

develop, release y hotfix

develop para trabajar, **release/*** para preparar versión, **hotfix/*** para urgencias y **main** solo con versiones.

+ Versiones claras y planificadas.

– Muchas ramas largas: en Fabric, muchos workspaces y más conflictos. conflictos.

TRUNK-BASED

todo a main, a diario

Ramas de horas, no de días. Lo que no está terminado se esconde con feature flags.

+ Casi sin conflictos.

– Pide tests automáticos y disciplina; disciplina; en Power BI cuesta.

Microsoft no exige ninguna. Lo que importa: **a prod llega el commit que probaste**.

En Fabric: un workspace, una rama

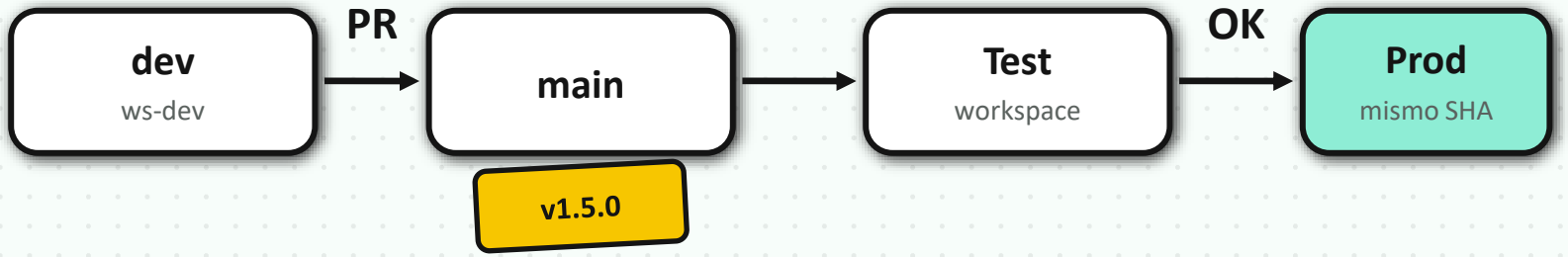
Dos formas de llevar Git hasta prod

A · UNA RAMA POR ETAPA



Cada etapa tiene su rama y su workspace. Se workspace. Se promociona con PR; si no va no va todo, con cherry-pick.

B · UN COMMIT CON ETIQUETA



Solo dev está conectado a Git. El tag lanza lanza fabric-cicd: primero test y, con aprobación, el mismo commit a prod.

Un workspace solo se conecta a **una rama**. La tercera vía no usa Git para desplegar: **deployment pipelines**, **pipelines**, después del descanso.

Reglas de la casa

Se configuran una vez y trabajan por ti

Protege main

1. Solo entra por pull request
2. Al menos una aprobación
3. Los checks, en verde
4. Sin force push ni borrados
5. Borra la rama al mergear

nombres que se entienden

```
# ramas
feature/medida-margen
fix/filtro-fecha
hotfix/kpi-ventas
release/v1.5.0

# commits
feat: añade la medida de margen
fix: corrige el formato de fecha
```

Ojo: en un repo privado de GitHub Free no hay protección de rama (hace falta Pro, Team o un repo público). En Azure DevOps, Azure DevOps, las branch policies vienen de serie.

Branch out, a fondo

Lo que ha cambiado este otoño en el panel Source control

Branch workspaces · GA

El workspace de la rama sabe de dónde sale: árbol de workspaces, miga de pan y pestaña de ramas relacionadas.

Selective branching

«Select items individually»: solo lo que que vas a tocar. «Select related» añade añade lo que necesita.

Add more items

¿Te faltaba uno? Source control → … → … → Add more items, y Update all.

Switch branch

Cambia la rama del mismo workspace. Pisa todos los items: commit antes.

Checkout new branch

Rama nueva sin perder lo que tienes a medias en el workspace.

Contributors al volante

Por defecto solo Admin cambia de rama. Un ajuste del workspace se lo deja a Contributor.

Preview: **branch out con deployment plan** y **perfil de admin** para branch workspaces.

MANOS A LA OBRA · 15 MIN

Ejercicio 5: una feature con su propio workspace

1. En GitHub, **workshop-git** → Settings → General: deja solo **Allow squash merging** y marca **Automatically delete head branches**.
2. En **ws-gitws-userN**, **Branch out** a un workspace nuevo, rama **feature/margen**, con **Select items individually**: el informe y **Select related**.
3. Haz dos commits pequeños: una tarjeta nueva (**feat:**) y el título de la página (**fix:**).
4. Pull request hacia main y **Squash and merge**: en main queda un solo commit.
5. En **ws-gitws-userN**, **Update all**. Después borra el workspace de la feature: vivía lo la rama.

Paso a paso: README del repo gitws-team

12:05–12:20

Descanso



Pásanos tu usuario de GitHub: te invitamos al repo común **gitws-team**.

Un conflicto no es un error

Dos personas cambian lo mismo y Git no sabe cuál vale. Así que te pregunta.



O juntas las dos a mano en GitHub y haces commit. **Si tocan partes distintas del fichero, Git las junta solo. las junta solo.**

Cinco reglas para no pisarse

Evitan casi todos los conflictos

1. **Update** antes de empezar a trabajar.
2. Ramas cortas: una rama vieja es un conflicto seguro.
3. Revisa leyendo el diff, no aprobando a ciegas.
4. ¿Conflicto? Se habla con la otra persona antes de elegir.
5. Repartid el trabajo: cada persona, su página o su item.

La 5 es la que más más importa en Power BI: dos personas en la misma página del del mismo informe informe es pedir un conflicto.

MANOS A LA OBRA · 15 MIN · LAS DOS JUNTAS

Ejercicio 6: el repo común

1. Crea un token **classic** con el scope **repo**: el del ejercicio 1 no sirve aquí.
2. En **ws-gitws-team**, **Branch out** a tu rama y cambia **tu** página del informe.
3. Commit y pull request hacia main: está protegida y pide una aprobación.
4. Revisa la PR de quien tengas al lado y apruébala. Luego, merge de la tuya.
5. Provocamos un conflicto a propósito y lo resolvemos en directo.

Paso a paso: README del repo gitws-team

Deshacer sin rezar

Con Git, volver atrás no da miedo



REVERT · SÍ

Un commit nuevo que deshace otro

El historial queda entero: se ve qué se hizo y que se deshizo. deshizo. Seguro en ramas compartidas.

RESET + FORCE PUSH · NO

Borrar la historia

Reescribe lo que otras personas ya tienen. En main, nunca. nunca.

En GitHub, una PR ya mergeada tiene un botón **Revert**: crea otra PR que la deshace. Luego, **Update** en Fabric. **Update** en Fabric.

MANOS A LA OBRA · 10 MIN

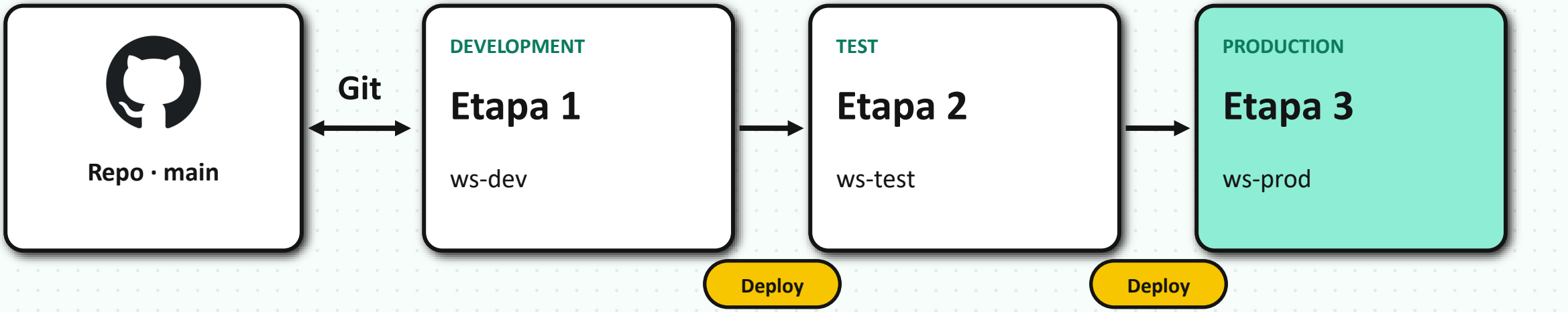
Ejercicio 7: deshaz tu cambio en el repo común

1. En GitHub, en **gitws-team**, abre **tu** pull request del ejercicio 6, ya mergeada.
2. Pulsa **Revert**: GitHub crea una PR nueva que la deshace.
3. Pide a quien tengas al lado que la apruebe, y haz merge.
4. En **ws-gitws-team**, pulsa **Update all**: tu página vuelve a como estaba.
5. Mira el historial de main: el cambio y su revert siguen ahí. Nada se ha borrado.

Paso a paso: README del repo gitws-team

Deployment pipelines: de workspace en workspace

Git llega hasta dev; de ahí en adelante, Fabric copia



De 2 a 10 etapas

Un workspace por etapa

Se copia la definición, no los datos

Lo bueno: sin YAML ni Python. Comparas, eliges y despliegas desde Fabric, y queda historial.

Qué viaja y qué se queda

Desplegar sobrescribe a la pareja del item, pero no a todo lo que la rodea

SE COPIA

La definición

- Modelo: tablas, medidas y relaciones
- Informe: páginas y visuales
- Notebooks, pipelines y el lakehouse (vacío)
- Las relaciones entre items y las carpetas
- Orígenes y parámetros (con reglas)

NO SE COPIA

Lo de cada etapa

- Los datos: el lakehouse llega vacío
- Permisos y ajustes del workspace
- Refresco programado y credenciales
- Quién está en cada rol de RLS
- La app de Power BI

Emparejamiento: al asignar el workspace, Fabric empareja los items por nombre y tipo. Al desplegar, cada item item sobrescribe a su pareja y esta conserva su ID, su URL y sus permisos.

Antes de darle a Deploy: compara

Cada etapa se compara con la anterior

Estado	Qué significa	Al desplegar
Same as source	Igual que en la etapa anterior	No cambia
Different from source	Alguien lo ha cambiado en una de las dos	Se sobrescribe con el de origen
Only in source	Item nuevo en la etapa anterior	Se crea en esta etapa

Completo

Todo lo de la etapa de origen.

Selectivo

Solo lo que marcas, con «Select related».

Hacia atrás

De prod a test, pero solo a una etapa vacía.

«Different» no dice quién ganó: si alguien tocó prod a mano, el despliegue lo pisa. En prod no se edita.

Lo que cambia en cada etapa

Reglas de despliegue y autobinding

Item	Regla que admite
Modelo semántico	Origen de datos y parámetros
Notebook	Lakehouse por defecto
Informe paginado	Origen de datos
Base de datos reflejada	Origen de datos
Dataflow Gen1	Origen de datos y parámetros

La regla se crea en la etapa de destino y se aplica en el siguiente despliegue.

Autobinding

El informe se reengancha solo al modelo de su misma etapa. etapa.

Ojo con Direct Lake

El modelo sigue leyendo el lakehouse de la etapa anterior. anterior. Hace falta una regla de origen de datos.

Variable libraries

En una etapa vacía llega activo Default: elige el conjunto de conjunto de valores de esa etapa.

Quién puede hacer qué

El pipeline tiene un solo permiso: Admin. El resto lo ponen los workspaces

Para…	Necesitas
Crear un pipeline	Licencia Pro, PPU o Premium
Asignar un workspace a una etapa	Admin del pipeline y Admin de ese workspace
Comparar y desplegar	Admin del pipeline y Contributor en origen y destino
Desplegar a una etapa vacía	Admin del pipeline, Contributor en origen y poder asignar la capacity
Crear reglas	Admin del pipeline, Contributor en destino y ser dueño del item

Quien despliega a una etapa vacía se queda como dueño de los modelos y como único Admin del workspace nuevo. workspace nuevo.

Git y deployment pipelines, juntos

El combo más habitual para empezar



Automatizable

La API de deployment pipelines, desde desde GitHub Actions o Azure DevOps. DevOps.

No toques test

Entre validar y desplegar a prod, nada nada cambia en test.

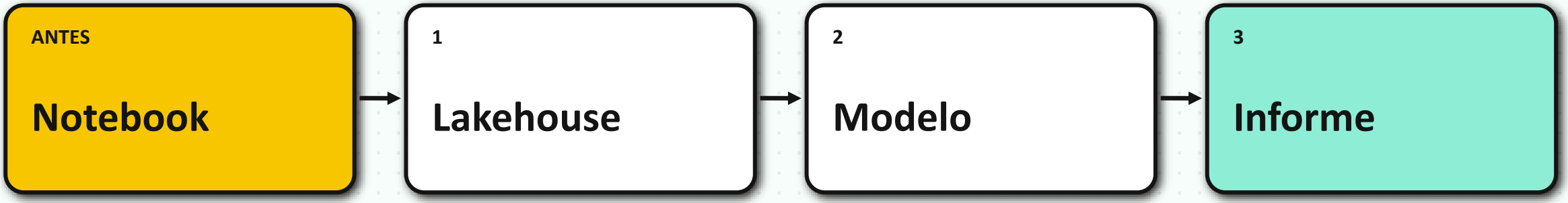
Nueva UI · preview

Todo desde la etapa: comparar, reglas e reglas e historial.

Y el orden, ¿quién lo decide? Deployment plans

Preview · FabCon Europe, septiembre de 2026

Un item nuevo del workspace que dice **en qué orden** se despliegan los items y **qué se ejecuta** antes y **ejecuta** antes y después.



Se adjunta al desplegar en un pipeline, al hacer Branch out o Update desde Git, y por API. No se lanza solo.
solo.

Ojo, es preview

No hay rollback: lo que ya se desplegó, se queda. Y fabric-cicd todavía no lo soporta.

MANOS A LA OBRA · 20 MIN

Ejercicio 8: tu primer deployment pipeline

1. **Create deployment pipeline** desde **ws-gitws-userN**: Development y Production.
2. Tu workspace queda en Development. **Deploy** a Production: Fabric crea otro.
3. Abre el workspace nuevo: el lakehouse está, pero vacío. Los datos no viajan.
4. Cambia el título del informe en Development: sale **Different from source**.
5. Despliega solo el informe, con una nota, y búscala en **Deployment history**.
6. Extra: mira el linaje del modelo en Production. ¿De qué lakehouse lee?

Paso a paso: README del repo gitws-team

Cuatro formas de ir de dev a prod

Las opciones de Microsoft. Se pueden mezclar

Opción	Fuente de verdad	Cómo se despliega	Lo que cambia por etapa	Para quién
1 · Una rama por etapa	Git: dev, test y prod	Git, fabric-cicd o API	Definición por rama o parameter.yml	Equipos que promocionan con PR
2 · Commit con etiqueta	Git: tag en main	fabric-cicd o Bulk import	parameter.yml y variable libraries	Equipos que sacan versiones
3 · Deployment pipelines	El workspace anterior	Fabric, con UI o API	Reglas y autobinding	Para empezar sin código
4 · ISV	Git: tag en main	fabric-cicd por cliente	Parámetros por cliente	Un producto, muchos clientes

Fuente: Microsoft Learn, «Choose the best Fabric CI/CD workflow option» (18/10/2026)

Pasar entre entornos

Variable libraries: un código, un valor por entorno

VARIABLE LIBRARY

lakehouse_origen
dias_historico
enviar_alertas

dev	lh_ventas_dev · 30 · false
test	lh_ventas_test · 90 · false
prod	lh_ventas · 730 · true

Un item más, versionado en Git.

Cada columna es un conjunto de valores y **cada workspace elige cuál tiene activo**. El código no código no cambia al pasar de dev a prod.

¿En qué peldaño estás?

La escalera de madurez CI/CD para informes, de Tabular Editor

hoy: de peltre a plata

PELTRE

Sobrevivir

- .pbix en OneDrive o SharePoint SharePoint
- Publicar desde Desktop
- El historial, el del fichero

BRONCE

Ordenarse

- PBIP y TMDL en Git local
- Commits pequeños
- Workspaces de test y prod
- BPA antes de publicar

PLATA

En equipo

- Git de Fabric con GitHub o ADO ADO
- Estrategia de ramas y workspaces
- Puertas de calidad en el pipeline

ORO

Automatizado

- GitHub Flow y rama de test desechable
- Validación, BPA y tests en cada PR
- Diff del modelo en la revisión

Fuente: Eugene Meidinger, «Building CI/CD maturity for report development», Tabular Editor (13/10/2026)

Puertas de calidad: si falla, no hay merge

Lo que hace un pipeline de plata u oro en cada pull request

paso de la PR · GitHub Actions o Azure DevOps

```
# BPA: rompe la build si hay errores
te bpa run --fail-on error

# El modelo y el informe están bien formados
te validate
pbir validate

# Tests de DAX y qué cambia respecto a main
te test run
te diff
```

Es lo que hacemos a ojo

Leer el diff y revisar, pero hecho por una máquina en máquina en cada PR.

Ojo con la IA

No dejes que un agente edite TMDL o PBIR a pelo: pelo: que use TE3, la CLI o el MCP de modelado. modelado.

Preview

La CLI de Tabular Editor está en preview limitada hasta el 30/11/2026.

Git y CI/CD en Fabric: lo nuevo

Septiembre y octubre de 2026

YA DISPONIBLE (GA)

Branch workspaces

Cada rama recuerda su workspace de origen.

Selective branching

Branch out solo con los items que necesitas.

Compare and commit

El diff dentro de Fabric, antes de hacer commit o Update.

APIs de export e import masivo

fabric-cicd ya las usa para publicar más rápido.

EN PREVIEW

File-level commit

Subir solo algunos ficheros de un item.

Perfil de admin para branch workspaces

Branch out sin crear workspaces ni asignar capacity.

Workspace Relation API

Crear por API la relación entre un workspace y su rama.

Nueva UI de deployment pipelines

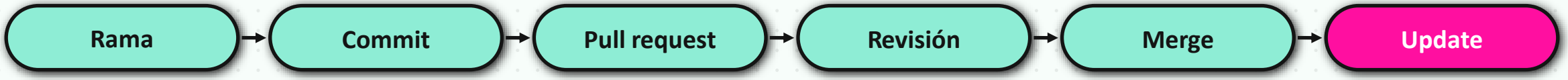
Todo desde la etapa: comparar, reglas e historial.

Desde el 1/12/2026: sin permiso de lectura y escritura sobre los items, no hay Git ni despliegue.

La chuleta

Hazle una foto

EL FLUJO



LOS BOTONES DE FABRIC · SOURCE CONTROL

Botón	Qué hace
Commit	Sube a Git lo que has cambiado en Fabric
Update	Trae a Fabric lo que ha cambiado en Git
Undo	Descarta lo que aún no has subido
Branch out	Rama nueva, con su propio workspace

LOS CUATRO HÁBITOS

- **Commits pequeños**
- **Di por qué**
- **Rama primero**
- **Nada de secretos**

¿Y el lunes, qué?

Cómo seguir sin nosotras

Tu repo es tuyo

workshop-git sigue en tu GitHub. Los usuarios y workspaces del taller se borran hoy.

Trabaja también en local

Power BI Desktop guarda como proyecto (PBIP). VS VS Code o GitHub Desktop hacen el commit sin terminal.

Llévalo a tu equipo

Conecta a Git un workspace de desarrollo y empieza por empieza por una regla: a main, solo por pull request. request.

Sube un peldaño

Mira la escalera de madurez y elige el siguiente paso. Uno, no cuatro.

¡Gracias!



Dime qué commiteas y te diré quién eres

Los ejercicios se quedan en el repo **gitws-team**. Tu repo **workshop-git** es tuyo: llévatelo.



Mar Lizana



Adrián del Rincón



Raquel Rega